

Command Development for the P?S Advanced PDP-8 Simulator Package.

©1975-1989, CHARLES LASNER ASSOCIATES

©1990-2015, CLA SYSTEMS

ClaSystems@gmail.com

Last edit: 05-Aug-2015 cj1

How to add a new program source file for development with the P?S Advanced SIMH package.

Additional topics include related issues with regard to OS/8 and P?S/8 as used with the SIMH PDP-8 simulator and resolution of problems where feasible.

Newer versions of this document may add additional topics as required. Feedback is welcome on any related topic. Contact Charles Lasner CLASystems@gmail.com for any questions.

Technical level: Moderate to high with respect to windows/MS-DOS as well as PDP-8 operating environments such as OS/8 and P?S/8.

The main purpose of this document is to explain how to organize the components needed to support easy access to the rest of this package for the purposes of developing a new source file project. Consequential issues that crop up during development will also be discussed.

Note: In very few instances, extremely long complete pathnames are indicated to be explicitly used as part of a command file or parameter. For readability purposes only, these are shown on multiple lines of this document with a § symbol. The actual usage must be one long line; a single <SP> character should be used where the § is shown.

Advanced SIMH Package Drive designation.

A valid volume and drive letter must be chosen to operate the P?S Advanced SIMH package. Depending on system complexity, this will be C: or any valid drive through Z: The choice will be set either manually within windows' environment variables section or using available batch command files within the overall P?S Advanced SIMH package. Due to the way various command lines function within MS-DOS and windows, the various shortcuts and batch files depend on a master environment variable for proper functionality.

Depending on the context, the string

%PDP-8_DRIVE%

is to be typed literally as stated. This applies to all command lines and input to shortcut fields, etc.

Advanced SIMH Drive textual reference.

When the reference to the drive is made as part of ordinary text within this document, it refers to the drive letter [and a ":" character] chosen for the actual drive used to install the P?S Advanced SIMH package such as D:. For example, the COROUTINE.PAL file described in the next section will be located on a volume on the target system such as drive D:. However, the path to all such files is described throughout this document starting with %PDP-8_DRIVE%. Since this is merely documentation, not the contents of a command file or parameter specification, mentally substitute the drive letter [and ":"] chosen for installation of the Advanced SIMH package.

Project files within the P?S Advanced SIMH environment.

Various source files can be developed within the P?S Advanced SIMH environment. To make this process significantly easier, various shortcuts and batch files can be created to avoid needless tedium.

As an example, we will use the existing file COROUTINE.PAL. [Note: COROUTINE.PAL is actually an excerpt from the source file for Kermit-12, a Kermit implementation for OS/8 written by Charles Lasner [with some help from others]. It was extracted as an example of a well-constructed practical coroutine for illustrative use in an academic setting to explain PDP-8 coroutines used in non-real-time programs.

The program segment is meant to be used to obtain unpacked bytes from an OS/8 input file called as a conventional subroutine from a main program; the coroutine aspect of things is totally contained within the called code including certain mode settings that could be set externally such as seven bit ASCII text or eight bit binary mode.

Non-real-time coroutines were chosen to introduce the general concept of coroutines to students who had no prior experience with this data structure; real-time usage of coroutines is a non-trivial matter. It is far more difficult to illustrate how coroutines facilitate foreground processing or inter-task communications without presenting a completely detailed entire program. The academic setting was meant to introduce students to coroutines, not cause them to run for the nearest exit.

The COROUTINE.PAL file is located within one of the recommended source file directories, in this case:

%PDP-8_DRIVE%\PDP-8\Source Files\Generic Source Files

which is used for source file projects not particularly associated with any particular operating system [such as OS/8 or P?S/8].

[Note: The environment variable PDP-8_DRIVE should already be setup by the package setup program as documented elsewhere. It should be set to D: or some similar drive letter other than C: if at all possible.]

Important: After first setting up the PDP-8_DRIVE environment variable, the windows system should be logged off or rebooted at least once before performing any of the operations described below to allow the environment variable change to take effect.

The objective is to create two batch files to support the source file, each with a shortcut to their respective batch file, such as:

- 1) Edit COROUTINE.PAL - This is a batch file that runs a designated editing program to modify the source file [which in this example is the windows standard Notepad.exe], and then pass the file to the SIMH PDP-8 simulator by creating a copy as the paper-tape input stream file called PTRFILE.TXT. [Note: Nearly all files used in the SIMH PDP-8 simulator use arbitrary names. By changing the content of the config.ini file[s], alternate file names can be defined to suit personal preference.]

If the source file editor is closed [which closes the batch file], the SIMH PDP-8 simulator can be run separately if desired; however, as long as the source file is updated, it is not necessary to end the editing session. [see 2) below.]

- 2) Copy and Go for COROUTINE.PAL - This is a batch file that also copies the file to PTRFILE.TXT as in 1) above, then runs the SIMH PDP-8 simulator to allow the high-speed reader handler of OS/8 to create the file within the OS/8 environment. As source file edits are made, this batch file can be used as need be.

The PDP-8 operating system of choice [OS/8 or P?S/8] is set within the batch file by setting the GOCONFIG environment variable as shown in the example below.

[Note: Future releases of P?S/8 will include the PTUTIL utility. This will allow P?S/8 to process passed input files directly. The present method is to first use OS/8 configured with paper-tape reader support; after the file is in the OS/8 environment, the P?S/8 OS8CON program can convert the OS/8 data into P?S/8 files.]

Example batch files.

Edit and Copy the COROUTINE.PAL file. The COROUTINE.PAL file is open in Notepad and then copied into the paper-tape input stream file PTRFILE.TXT afterwards. When the editing session completes, it would be appropriate to run the SIMH PDP-8 simulator booting the operating system of choice [OS/8 or P?S/8].

File Name: Edit and Copy COROUTINE.PAL.BAT

```
REM Edit a specific file then copy it into the PTR: file in
REM preparation for assembly in SIMH.

REM Last edit: 05-Aug-2015 cjl

REM Set the file name.

SET FILENAME=COROUTINE.PAL

REM Set the path for source files.

SET SRCPATH=%PDP-8_DRIVE%\PDP-8\Source Files\Generic Source Files

REM Set the path for the working files.

SET WORKPATH=%PDP-8_DRIVE%\PDP-8\SIMH\PDP-8\SIMH_working_Files

START /WAIT NOTEPAD.EXE "%SRCPATH%\%FILENAME%"

REM Copy the source file to the SIMH PTR: file.

COPY /Y "%SRCPATH%\%FILENAME%" "%WORKPATH%\PTRFILE.TXT"
```

For a new source file, the FILENAME and SRCPATH lines must be changed. Alternatives to Notepad.exe can be used [specifying an explicit path as required].

The next batch file will copy the COROUTINE.PAL file to the paper-tape input stream file PTRFILE.TXT. It is presumed a host editing session might be in progress which need not be terminated. The SIMH PDP-8 simulator will be run booting to either OS/8 or P?S/8 as determined by the setting of the GOCONFIG environment variable within the batch file.

Eventually, after looking at results obtained from the SIMH PDP-8 simulator session [such as PTPFILE.TXT or LPTOUT.TXT after appropriate filtering and stripping has modified the file to be acceptable to the host environment], further editing can be performed without necessarily terminating the editing session.

Note: OS/8 users have a choice of which output stream to pass the results back through. P?S/8 users should use LPTOUT.TXT due to limited support of the high-speed punch within the present release of P?S/8 associated with the Advanced SIMH PDP-8 simulator package at the time of this writing.

The developers of P?S/8 have always avoided paper-tape hardware support for a variety of reasons:

- a) Using paper-tape is incredibly inefficient and was wasteful on the actual computer hardware used back in the day,
- b) No high-speed paper-tape hardware was available to the P?S/8 developers; only the slow and quite unreliable Teletype Model 33 ASR was available on the specific PDP-8 systems used to test and develop P?S/8.

In fact, the original system used for testing and development had a Teletype Model 35 KSR as the system console; as such, no paper-tape capability existed on that system.

Future releases of P?S/8 will include the PTUTUL program supporting ASCII text paper-tape devices including the device 01 high-speed reader and device 02 high-speed punch, and the Teletype Model 33 ASR and Model 35 ASR console terminals which can read and punch paper-tape at low speed only.

The paper-tape support is being added primarily as a concession to the SIMH PDP-8 simulator environment; the high-speed paper-tape reader and high-speed paper-tape punch is the primary device used to bring files in and out of the simulated environment to allow host utilities to operate on the files as well as using PDP-8 facilities. PTUTIL will allow the reading and creation of P?S/8 TFS and extended-length files when completed. [Note: PTUTIL for P?S/8 Version 8P may not support extended-length files due to the primitive suport inherent in this early release.]

File Name: Copy and Go COROUTINE.PAL.BAT

@ECHO OFF

REM Copy the source file to the SIMH PTR: file and then run SIMH.

REM Last edit: 05-Aug-2015 cjl

SETLOCAL

REM Set desired PDP-8 bootup file below; REM the other one.

SET GOCONFIG=os8config.ini

REM SET GOCONFIG=pqsconfig.ini

REM Set the file name.

SET FILENAME=COROUTINE.PAL

REM Set the path for source files.

SET SRCPATH=%PDP-8_DRIVE%\PDP-8\Source Files\Generic Source Files

REM Set the path for the working files.

SET WORKPATH=%PDP-8_DRIVE%\PDP-8\SIMH\PDP-8\SIMH_working_Files

REM Copy the source file to the SIMH PTR: file.

COPY /Y "%SRCPATH%\%FILENAME%" "%WORKPATH%\PTRFILE.TXT"

REM Now run the SIMH PDP-8 Simulator. If running OS/8, read the PTR:
REM into a .PA or other text file using OS/8 PIP. If running P?S/8,
REM use PTUTIL to read the PTR: input and create P?S/8 output files.

CD "%PDP-8_DRIVE%\PDP-8\SIMH\PDP-8\SIMH_working_Files"

"%PDP-8_DRIVE%\PDP-8\SIMH\PDP-8\SIMH Simulator Releases\PDP8.exe"

§ "%PDP-8_DRIVE%\PDP-8\SIMH\PDP-8\SIMH Configuration Files\%GOCONFIG%"

Note: The last file line is shown in two parts separated by § for readability purposes only; it should be one continuous line with a single <SP> character between the two sections.

For a new source file, the FILENAME and SRCPATH lines must be changed.

Both batch files should be stored in the

%PDP-8_DRIVE%\PDP-8\SIMH\PDP-8\SIMH Batch Files

directory to support the new source file.

Creating proper shortcuts for the new source file.

Example shortcuts for the batch files relating to COROUTINE.PAL must contain the following:

Shortcut name: Edit COROUTINE.PAL.lnk

Important Properties:

Target:

"%PDP-8_DRIVE%\PDP-8\SIMH\PDP-8\SIMH Batch Files\Edit and Copy
COROUTINE.PAL.BAT"

Start in: "%PDP-8_DRIVE%\PDP-8\SIMH\PDP-8\SIMH Batch Files"

Run: Minimized

Screen buffer size:

width: 132

Height: 1000

window size:

width: 132

Height: 132

[Note: Colors are unimportant for this shortcut because the window is run minimized.]

When setting the icon for the shortcut, choose the following path in "Change Icons":

"Look for icons in this file:"

%PDP-8_DRIVE%\PDP-8\SIMH\Links and Support Files\Desktop Shortcuts and Icons\question.ico.

The actual icon file chosen in this directory is a user preference. However, always choose an icon from this directory. [Additional icons can be added to this directory as needed.]

After all of the parameters of the shortcut are set, open a command prompt and navigate to the %PDP-8_DRIVE%\PDP-8\SIMH\PDP-8\Commands directory. Set the file attributes as follows:

ATTRIB [*named shortcut .lnk file*] +a -h +s +r [Note: All pre-existing files should already have the attributes set this way. Use the ATTRIB command without arguments to see the current status of all .lnk files.]

Note: The reason for the attribute change is to prevent windows from attempting to "guess" for new values of any of these parameters should something be set wrong, such as failing to set the PDP-8_DRIVE environment variable as directed. Without the environment variable in effect, some of these parameters cannot be set. By locking down the parameters, copies of the P?S Advanced SIMH PDP-8 simulator package will not be accidentally modified. Should there be a reason to change any parameter, the read-only attribute of the shortcut can temporarily be changed from within the properties of the shortcut itself to allow updating. After all changes are performed, ensure the attributes are set as documented here.

Note: windows does not provide a graphical interface to control the System attribute bit; only a command prompt can accomplish this. Note that .lnk files with the System attribute bit will not show in a DIR command at an MS-DOS prompt; however, these files will be revealed by the ATTRIB command as well as in windows as icon/link properties.

Example shortcut links.

Shortcut name: Copy and Go COROUTINE.PAL.lnk

Important Properties:

Target:

"%PDP-8_DRIVE%\PDP-8\SIMH\PDP-8\SIMH Batch Files\Copu and Go
COROUTINE.PAL.BAT"

Start in: "%PDP-8_DRIVE%\PDP-8\SIMH\PDP-8\SIMH Batch Files"

Run: Normal window [Note: Maximized should be used
with caution; Normal window can
generally be adjusted to nearly
full screen if desired.]

Screen buffer size:

width: 132

Height: 1000

window size:

width: 132

Height: 132

Color options for this shortcut are significant and are a user preference. The values chosen in Copy and Go COROUTINE.PAL.lnk will produce a window color scheme for the SIMH simulator reminiscent of a roll of teleprinter paper as would typically be used on a Teletype Model 33.

The icon path/choice and attributes considerations are the same as in the previous example.

Both shortcuts should have Display options set to "window" to allow the window size to be adjusted during use [which is the default setting].

It is recommended these two example shortcuts be copied to become templates of the new shortcuts and then modified to match the intended new file. This will cause most of the settings to be either set properly or be fairly close. Note: The copies must have the Read-only attribute removed to change anything; when done, set the Read-only attribute as described above. Do not attempt any of these changes unless the %PDP-8_DRIVE% environment variable is already set and the current profile has been completely logged off at least once [or the system is rebooted].

Note: Inactive shortcuts can be removed from the
%PDP-8_DRIVE%\PDP-8\SIMH\PDP-8\Commands directory
and placed into the

%PDP-8_DRIVE%\PDP-8\SIMH\PDP-8\Inactive Commands

directory to avoid clutter in the main Command window. The other shortcuts in the Command directory can be studied in case additional utilities are contemplated. The examples here are oriented towards adding a new project file to the working copy of the PDS Advanced SIMH PDP-8 simulator package. The other provided shortcuts and underlying batch files [if relevant] are a sufficient set of utilities for most users; studying the existent files should be an adequate guide to anyone wishing to implement additional capabilities. A thorough working knowledge of windows batch files and directories is recommended before proceeding as there are nuanced issues that need to be addressed for a successful addition to this package.

Please contact Charles Lasner [CLASystems@gmail.com] for any relevant issues or additions.

Passing files from the host environment to the SIMH PDP-8 simulator [and back].

Assuming the file is setup by a Copy and Go routine, the following should be executed within the SIMH PDP-8 simulator OS/8 environment [the COROUTINE.PAL file is the subject of this example]:

```
.R PIP
*COROUT.PA<PTR:
^[Press any key to "start" the "reader".]*
```

Note: OS/8 only allows six character names with two character extensions in upper-case. When the P?S/8 SHELL is completed and a similar utility is available, files will be allowed with up to 12 character names with up to four character extensions. The file name case will be retained for all cosmetic purposes but will be otherwise treated as caseless. Thus, in the P?S/8 SHELL environment, the entire file name could generally be used without truncation.]

The file can then be used in the OS/8 environment and edited with TECO.SV if desired. PAL8 can be used to assemble the file and create a listing output file. A typical usage might create COROUT.BN as the binary file and COROUT.LS as the listing file intended for printing or other purpose.

Any file created within the OS/8 environment can be brought into the host environment as a file:

```
.R PIP
*PTP:<COROUT.LS
```

When the SIMH PDP-8 simulator is closed, utilities can remove artifacts making the resulting file acceptable to the host windows/MS-DOS standards.

Only one file at a time can be returned to the host environment. As such, it may be desirable to only edit in the host environment. TECOC for windows is available within the SIMH PDP-8 simulator Utilities directory; it is used as part of the filtering utilities needed to remove artifacts.

The editing method used in this example involves the use of the standard Notepad.exe program within Windows. All non-printing characters other than the exact sequence `<CR>` immediately follows by `<LF>` show as a thin rectangle with no way to distinguish the exact character. Listing files will show pages separated by the `<FF>` character in this manner. The character can be copied and pasted within Notepad to insert characters where desired; however, it is the user's responsibility to ensure that no other non-printing characters are present. Editors such as SEDT or Notepad++ may be used if desired to reveal the codes of characters accidentally placed in the file. Appropriate shortcuts should be created in lieu of those described above if desired.

Optional P?S/8 development.

Also provided in this package are releases of P?S/8 circa 1976 [Version 8P] and 1989 [Version 8Z] with minor binary patches applied in 2015 to fix longstanding bugs until a new source code-based release is available tentatively to be known as Version 9A. In certain ways, P?S/8 is superior to OS/8 [and admittedly there are tradeoffs between the systems presently]. For more information about issues related to OS/8 and P?S/8 [including historical information about early development of both including the R-L Monitor] consult Appendix A of this document.

Note: P?S/8 is currently being developed using the SIMH PDP-8 simulator package as described in this document. Updated files will be made generally available as the author sees fit; this will include interim updates as well as complete releases [or patched versions] as they occur.

P?S/8 source files are maintained in the host environment and presently assembled in OS/8. Once a rudimentary implementation of the P?S/8 SHELL is completed, further development will be performed entirely in the SHELL environment [without OS/8].

Source files may be freely converted from the OS/8 environment to/from P?S/8 using prevailing conversion programs in P?S/8. More documentation is available elsewhere regarding file conversion both in the present system and in future releases of both the basic P?S/8 system and the optional SHELL environment.

Note: The conversion programs depend on specific configurations of the PDP-8 as carried out within the SIMH PDP-8 simulator configuration as used in this package. Where possible, the normal handlers associated with P?S/8 will be compatible with OS/8 at the physical block level.

In certain cases, it will be necessary for the P?S/8 SHELL to use special handlers geared towards OS/8 access to certain devices where P?S/8 uses a different method of access [generally to provide more storage on the same hardware; many PDP-8 peripherals can be used in a manner that wastes 1/3 of the available space. OS/8 generally uses the inefficient operational modes on these devices due to lack of proper code space within the meager OS/8 handlers. P?S/8 handlers may be provided for either mode depending on hardware and configuration specifics. Only certain specific inter-system configurations will be acceptable to the conversion programs. When the SHELL conversion program is written [tentatively to be known as PIPOS8] certain configuration rules can be somewhat relaxed.]

File Conversion quirks and remedies.

P?S/8 ASCII text paper-tape issues.

P?S/8 was written in an environment devoid of paper-tape, universally despised by all serious PDP-8 programmers. Yet, in the SIMH PDP-8 simulator environment, the only way to readily communicate files between the SIMH simulated PDP-8 and the host environment is the use of the high-speed reader and punch handlers simulated as the contents of two files in the host environment.

Until a P?S/8-based ASCII paper-tape utility is written [tentatively to be named PTUTIL], it will be necessary to use OS/8's high-speed reader [and perhaps punch] device handlers to bring source files into and out of the SIMH environment for further editing and other work in the host environment [Windows].

Each available implementation has a unique utility for conversion between OS/8 and P?S/8. The companion P?S/8 command guide document details the method available to both systems.

Using the 1976 release of P?S/8 [Version 8P], the HR{eader} command can be used to create P?S/8 TFS text files directly; unfortunately, the lengths of the files must be carefully controlled to avoid loss.

Note: The HR{eader} command was dropped in later implementations of P?S/8.

When the P?S/8 SHELL is implemented, utilities similar to PTUTIL will be available for a more direct conversion process.

P?S/8 binary paper-tape issues.

P?S/8 supports binary paper-tape data using built-in command-line options of the BIN utility. [The exact functionality is determined by proper selection of command-line option switches; BIN program documentation is available in other documents.] This includes the following capabilities:

- 1) Loading existing binary TFS files such as are created by assembling source programs with P?S/8 PAL; various loading and startup options are set by command-line options.
- 2) Launching a [slightly modified] BIN [and RIM] loader with built-in system reboot by restarting at 07600. Both the low-speed and high-speed reader are supported.

Note: The modifications to BIN are consistent with the usual implementation of the RIM loader; the command-line options determine which reader device is used in both loaders; no purpose is served using the front panel switch register to select the input device since the BIN and RIM loaders are being launched from a storage-based operating system. This is especially helpful in the SIMH PDP-8 simulator environment since the switch register must be set by unwieldy keyboard commands that suspend the SIMH simulator.

Unlike the DEC version of the BIN loader, P?S/8 can be restarted at 07600 while the BIN and RIM loaders are in memory [assuming the bootstrap code is not destroyed by the program loaded by BIN or RIM].

Note: Restarting P?S/8 replaces the contents of 07600-07777 with the P?S/8 system kernel which, in turn, loads in the P?S/8 keyboard monitor and integrated editor. The BIN and RIM loaders are provided merely as an accommodation to stand-alone paper-tape loading when [infrequently] needed; this is necessary to load "rogue" programs known to violate PDP-8 operating system conventions such as several DEC PDP-8/E diagnostic programs. For all conforming programs, loading should be accomplished within P?S/8 by loading binary files on the storage device.

As with all other versions of the BIN loader, the standard restart address of 07777 applies; all known versions of the RIM loader must be restarted at 07756; the specified loader [RIM or BIN] is automatically started by executing the command-line with appropriate loading options. The BIN utility supports several user memory pre-loading options using command-line options that can be performed before starting the selected loader.

- 3) Reading binary paper tapes and converting to multiple binary TFS files including support for multiple sections within a single tape. Both the low-speed and high-speed readers are supported in a choice of BIN or RIM format.
- 4) Reading existent binary TFS files on the storage device and punching the contents to the low-speed or high-speed punch in either BIN or RIM format.

Note: This implementation of RIM format includes a checksum allowing the tapes to be read using either BIN or RIM loaders.

OS/8 paper-tape and line-printer handler issues.

The OS/8 paper-tape output handler exhibits bizarre quirks creating unexpected conversion difficulties.

Note: documentation of part of this problem is so complex there is under development a separate "white paper" document on this problem. The problems exist by design despite wide-spread ignorance of the problem. [The problems are quite apparent in the host environment when outputting ASCII text files through the PTP: handler.]

Certain of these problems are shared by the OS/8 device 66 standard LPT: handler; some of the problems actually originate in the system programs used; the LPT: handler can magnify the severity of the problem.

P?S/8 line-printer issues.

P?S/8 does not output to the paper-tape device [other than binary files]. The standard device 66 line-printer is used, which also creates several quirks.

Note: Some of these problems are considered bugs associated with the 1976 release of P?S/8 fixed in later releases.

Fixing problems using file filtering.

All of the problems stated above [and more] have been overcome through the use of a filtering utility based on TECOC [TECO for windows and MS-DOS] and TECO macros written by Charles Lasner. Shortcuts to the utility exist in the Command directory of the P?S Advanced SIMH PDP-8 simulator package; both paper-tape and device 66 line printer output filtering are supported.

Filtering details and issues.

The basic purpose of filtering output files is to strip the high-order bit of all ASCII text characters since the host environment does not tolerate mark parity set; the host environment reserves this bit for output characters beyond the scope of both P?S/8 and OS/8.

However, there are a variety of additional problems beyond this [some of which were totally unexpected]. Some logistic details of certain discovered problems follows:

- 1) A widespread problem of many programs in both operating system stems from the inconsistent use of `<CR>` not followed by `<LF>` or `<LF>` not preceded by `<CR>`. Additionally, other context-dependent problems exist that must be handled in the proper order of discovery after these problems are noticed.

Note: Certain fixes are extremely pragmatic.

- 2) Several "varieties" of unexpected artifacts are present in the output of PAL8 [and to a far lesser extent the output of P?S/8 PAL also exhibits certain additional quirks]. Pragmatic code in the filtering utilities handle all known quirks in the output associated with both operating systems.

- 3) Once all detected artifacts are removed from a PAL8 output file, any leading `<FF>` character[s] will be removed. Modern printing techniques performed in the host environment include various forms of post-processing; initial blank pages will be eliminated.

Note: In past years having an initial `<FF>` made sense: a) it is not possible to determine the prior state of the line printer, b) the previous page may not be set to top-of-form and c) there could be a partially printed page that must be ejected before starting the current print job. Print management has been transferred to the host operating system to handle all such printing issues during post-processing operations beyond the SIMH PDP-8 simulator environment.]

- 4) The opening content of the 1976 P?S/8 PAL output files is a bit more complicated than merely a single `<FF>` character [after removal of all quirky output in the files created by both operating systems]. As of this writing, the latest filtering utilities remove the specific sequence; thus, there is basic parity in the output of P?S/8 PAL as compared to the output of PAL8. [Update: The output of 1988 P?S/8 PAL is now supported to the same extent; the opening content was changed and unified. What is removed makes sense when dealing with real-world printers; however must be removed for host printing purposes.]
- 5) The final `<FF>` and related artifacts are also removed from all P?S/8 PAL and PAL8 output. This is a bit more complicated than expected as both assemblers have quirks in the output as a function of specific options selected. Pragmatic code in the filtering utility handles all known cases.

Note: In the unlikely event a file contains more than a single trailing `<FF>` character, only the last one is removed. Repeating the filtering operation on the file [as many times as required] will remove additional trailing `<FF>` characters. PAL8 is especially prone to this when the output is sent to the high-speed punch handler specifically; no case of more than a single extra trailing `<FF>` character has been observed, but the filtering program is prepared to handle as many as necessary by repeating the filtering process.

Note: It is theorized the presence of all of these problems is associated with bad documentation and limited contact with line printers in general by the PDP-8 programmers involved with both operating systems. Some output techniques used were merely mimicking poor output routines written by early hardware engineers.

The SIMH PDP-8 simulator environment tends to shed light on all programs using bad techniques. However, some additional problematic "features" were added to the OS/8 PTP: handler for dubious purposes; this will be documented in an external "white paper" document as mentioned above. Regardless of reason, the filtering utilities remove all known problematic cases.

Additional issues.

OS/8 is not capable of producing a direct output of the BITMAP program as part of an integrated assembly/map operation. However, the separate output of the BITMAP program can be concatenated with the main PAL8 listing file for final printing or post-processing [such as creating a composite .PDF file].

P?S/8 supports the MAP program [a superset of the OS/8 BITMAP program] which can be chained to by P?S/8 PAL using command-line options. [In one operation, it is possible to assemble, map, load and go as long as there are no assembly errors.] As such, the LPT: output from both programs is produced in the same session and will be filtered along with the P?S/8 PAL output.

Various additional output quirks appear in the MAP output only; as of this writing, the filtering utility handles all known problems. [Some of the problems were remedied in newer releases of the MAP utility.]

All trailing <FF> characters are removed from all output; this is consistent with host management and post-processing techniques that apply.

For those curious to see these problems directly, the P?S Advanced SIMH PDP-8 simulator package allows viewing the line printer and paper-tape output files; viewing a file before applying the filtering utilities will illustrate what problematic quirks exist within the file [and then viewing after application of the filtering utilities will show how the output was corrected].

P?S/8 Date issues.

The basic P?S/8 file system [known as the TFS or Tiny File System directory] does not support individual file dates. However, the operating system supports an overall date that ranges from 1960 to 2087 in the original implementation.

Note: The date algorithm and base year are being modified for the upcoming release of P?S/8 Version 9A. The date support is being expanded to handle all dates from 01-Jan-1900 through 31-Dec-2411. The former modestly tedious maintenance requirement will be completely removed; the entire date will be available to any conforming program directly within the system kernel.

In the 1976 implementation of P?S/8 system date support, the low-order portion of the date is implemented as a 12-bit word that can be set by the system DATE utility. Programs such as P?S/8 PAL and P?S/8 MAP access the current value to provide date support in listing output.

However, the high-order portion of the date is implemented by pragmatisms within each program to be coordinated as part of a system maintenance to be performed manually. Thus, for each program that supports the current date, it will be necessary to perform a minor patch to specific locations within the core-image of each program.

Unfortunately, the 1976 release was the first implementation of the date support; it is not known whether the date can be updated past 1976 through 1987. There is no surviving documentation on specifics of this release.

The 1988 release of P?S/8 [Version 8Z] uses the same method of date support and is fully documented. The release associated with this document has been updated as described below:

Each program includes an internal storage location for the date year base. There is no particular pattern as to where the internal location is located since the matter is generally handled by program initialization code.

Note: An associated change log document is part of this package. All binary patches are logged detailing the exact locations modified; all date-related patches can be found there for future use. [The next update can be performed 01-Jan-2016; however, it is hoped version 9A of P?S/8 will be available by then.]

The value stored in the date year base word is multiplied by eight. This becomes the high-order portion of a double-precision calculation of any day in the entire date range relative to 01-Jan-1960.

For example, in 1960 the value would be 0, in 1968 it would be 1, etc. For the current base year of 2008, the value should be set to 6.

The date can be set to any day in the range of the base date through the end of the eleventh year past; thus, the current release of P?S/8 Version 8Z will allow valid dates from 01-Jan-2008 through 31-Dec-2019.

Once the P?S/8 system kernel is updated to support the entire date range in memory, all date-related source code will be modified to conform to the new values; this entire issue will then be obsolete for users of P?S/8 Version 9A.

OS/8 Date issues.

OS/8 has a series of unfixable date problems:

- 1) The internal date within OS/8 is limited to 1970-1977. However, a kludge exists to allow the date to be **perceived** to be as late as 2001 [assuming that certain program bugs are fixed; as such, certain outputs will become corrupted if the year is set past 1999].

The underlying problem is that OS/8 stores a 12-bit date word found in memory from the original pre-PS/8 design that was never updated.

Note: Some users disable the file date entirely because the directory structure is so compromised that the underlying storage may become inaccessible as a function of the total number of files and empty areas on the device; adding an additional word per file entry reduces the overall number of file entries allowed in the directory. This is a device-wide setting; you must give up all file dates to regain the space in the directory for more files [or empty entries].

In a hasty desperation measure implemented in 1977 after several years of procrastination, two additional bits were made available for the overall date; however the additional bits are implemented in memory only. As such, the date can be set to any year in the range of 1970-2001. However, assuming there is a date word function in the directory, the best that can be displayed is the **presumption** that all files are created in the year range of the current year and the previous seven years regardless of any differing reality.

worse still, this means that files for the present year will appear to be in the future if they are for an apparent date later in the present year than today. In reality, the file was created either 8, 16 or 24 years in the past, with no way to determine which specifically applies; the fantasy presentation of files created in the future would have to be ignored.

- 2) The entire kludge breaks down in 2001, the last year the kludge can be operated. Due to program bugs, certain programs will mishandle dates past 1999.

Note: P?S/8 has been tested well past 2000 for format errors associated with the end of a century; OS/8 suffers from the infamous "Y2K" problem.

Date Issue Recourse.

It is recommended that files in both the 1976 release of P?S/8 [Version 8P] and OS/8 be set to 1986. Files that are created inside of the SIMH PDP-8 simulator environment can be transmitted to the host environment with these dates. The values can be edited to current years for cosmetic consistency.

Other than proper date patch maintenance, there are no such problems in P?S/8 Version 8Z. Future releases of P?S/8 will require nothing but manually updating of the current date as necessary.

Note: Unlike OS/8, P?S/8 retains the current date until changed.

Other than changing the date for cosmetic purposes, it is expected the filtering utility will correct all output file quirks created by any of the three supported systems. The file dates can be updated with trivial TECO macros to find and replace all date strings with more appropriate values as needed.

Example:

The date field in an assembler output contains the date string "04-10-86" on every page; a corrected date might be "04-10-15". The following is an editing session that can be run in the host environment from an MS-DOS prompt.

Note: All TECO-related documentation uses \$ as a stand-in for the <ESC> character as TECO will output \$ when ESC is typed.

```
C:\>TECOC
*EBLPTOUT.TXT$<FN04-10-86$04-10-15$;>EFEX$$
```

From the C:\> prompt TECOC is executed. The TECOC prompt is *. Type in the command exactly as stated above [substituting the desired file to be modified; in this example LPTOUT.TXT in the current directory will be modified].

Note: the correct current date string contents must be provided, as well as the appropriate replacement text; overall current format rules must be obeyed to avoid unexpected output file aberrations.

- 1) EBLPTOUT.TXT\$ This opens the file LPTOUT.TXT in the current directory for editing with a backup file which will be named LPTOUT.BAK.
- 2) <FN04-10-86\$ Perform a non-stop search of the file attempting to locate the string 04-10-86. All non-matching characters are processed and placed into the output file unchanged.
- 3) 04-10-15\$ This is the replacement string used for every matching string.
- 4) ;> TECO commands placed within < and > run in infinite iteration loops. When the search fails, the iteration is broken and control passes to what follows >.
- 5) EF The output file is closed; the original file is retained with the .BAK extension.
- 6) EX TECOC exits to the operating system.
- 7) \$\$ All commands to TECO must end with ESC ESC. This allows <ESC> to be placed within the command. When the second ESC is typed, the entire command executes; until committing the command, errors can be corrected by pressing the Backspace key as many times as necessary, then typing the rest of the command.

At that point, candidates for output can be post-processed in either Microsoft word using an appropriate font [such as M+ 1m, a freeware monospace font capable of creating output similar to classic Centronics printer output as used by DEC], or the DaVince PDF package can be used to produce a greenbar output reduced to fit on 8.5x11 paper in landscape mode.

Note: PDFSAM [PDF Split And Merge] can be used to create mixed output as a single .PDF file for specific packages of program listings, documentation and cover pages. It can also be used to remove artifacts that may produce superfluous pages in the output.]

Appendix A.

A few comparisons of OS/8 and P?S/8.

Most of the information in this section is based on the vast experience of the author of this document, Charles Lasner who is the principal developer of P?S/8. He has vast experience with OS/8 and has written utilities for OS/8 most notably the Kermit-12 package for the OS/8 family for all PDP-8 models, the PDP-12 and all DECmates. One of his specialties is writing device handlers for both P?S/8 and OS/8 for various compatible devices sold by third-party companies for various 12-bit computers such as the MDC8 from Computer Extension Systems, Inc [CESI] [including all MDC8 system diagnostics]. Few individuals are in his unique position to provide an informed critique of both operating systems including both shortcomings and strong points where applicable.

As of this writing, OS/8 is a totally dead system; no one is developing any system-wide improvements; few, if any, are even possible due to original poor design and management interference. At best, we can use it in many ways as an example of what *not* to do.

P?S/8 is currently under active development after a period of several decades of stagnation due to personal issues of the author, sufficiently overcome at this stage to allow active pursuit of advancing this system that actually predates OS/8 by several years.

Early history of the R-L Monitor System, OS/8 and P?S/8.

P?S/8 is based in part on the R-L Monitor System which predates all single-user PDP-8 DEC operating systems of any importance. It was not well received by DEC management, but was universally loved by most of the DEC PDP-8 programming group, many of whom met at Brooklyn Polytech in the 1960's and 1970's.

The R-L Monitor system was written by some of the best PDP-8 programmers ever, most notably Richard Lary and Lenny Elekman. Their numerous accomplishments include POLY BASIC, later sold by DEC as EDU-System 30 and EDU-System 15/30 [for the TD8E only] and various other projects. Individually, Lenny Elekman is the author of 4K BASIC, which is the basis for 8K BASIC, various components of OS/8 BASIC in conjunction with others, and FOCAL-12 for DIAL-MS. DIAL-MS was written by one of their colleagues Jack Burness, who in developed 8K BASIC from 4K BASIC. These extraordinary programmers also worked with their friend and colleague Hank Maurer, the original developer of DIBOL-8, the precursor of COS-300/310.

Richard Lary is also the original producer of the demo version of what later became PS/8 [later renamed OS/8 because marketing complained about having to sell a "programming system" instead of an "operating system" thus more appropriate system name, at least according to the salesmen of the time.]

Appendix A [continued].

Although never officially acknowledged, it is true that the R-L Monitor System with hacked-up command names was the actual release of the original DIBOL-8; the only new contribution was the DIBOL compiler and run-time system added by Hank Maurer. The command changes were made to be more compatible with the paper specification of COS-300 which was not quite yet under development.

Clearly, the R-L Monitor System plays an important role in the history of DEC operating systems, even if not documented outside of the small circle of PDP-8 programmers who were personal friends of each other.

OS/8 is a mediocre system that was demonstrated to DEC management by Richard Lary as a weekend hack, but was a prematurely fixated design with many shortcomings; as such, it is mediocre at best and eventually much of the design limitations caught up with it. There are problems such as poor directory design and an unworkable date methodology that ran out in 1977, but was kludged to appear to work to the uninformed until perhaps as late as 1999 or in some cases 2001 depending on how many fixable bugs were still present.

It is most important to know that OS/8 as written was intended only as a demo for another system to be written from scratch, including a lot of input from the prevailing PDP-8 expert community at the time, including Charles Lasner, who was asked to work on the project and refused; instead he worked with other well-known PDP-8 expert programmers on making improvements to the R-L Monitor System.

Note: The week-end hack that eventually became OS/8 was known to PDP-8 programmers as the First Upward-Compatible Keyboard Editor and Monitor [with the obvious somewhat offensive acronym]. It was to be a demo for a true replacement system based on PDP-8 community input to be known as the Second Upward-Compatible Keyboard Editor and Monitor [with the similar acronym]. Had the second system been designed and implemented, it is entirely possible that Charles Lasner would have been one of its architects. [In certain ways, the emerging P?S/8 SHELL project is the fulfillment of this never-developed product.]

Had Richard Lary been treated at the time as the expert he is, OS/8 as we know it would have been totally scrapped. However, DEC management has always been the weak spot [and ultimate downfall] of the company.

Most notably, of the five original PDP-8 pioneering experts [Richard Lary, Lenny Elekman, Jack Burness, Hank Maurer and Joe Fischetti], all of the people on the list other than Richard Lary refused to work on the project Richard Lary was virtually forced to continue despite it not really being his intention either. [Charles Lasner was asked after the others refused to deal with it; he sided with the majority.]

Appendix A [continued].

Despite widespread conventional wisdom to the contrary, Richard Lary's participation in OS/8 virtually ended early on, other than to periodically participate with others on certain projects such as TECO for OS/8 and Fortran IV's runtime system to allow simulation of the FPP hardware when necessary. Unfortunately, most of the work was done by mediocre people and thus OS/8 is largely a mediocre system based on design elements never really thought through.

In many ways, OS/8 was figuratively painted into a corner due to the poor design of both the directory and the basics of the system resident kernel that are unfixable without essentially starting all over; this has been deemed not worth bothering with by prevailing PDP-8 developers such as Charles Lasner.

As the list of PDP-8 developers dwindles to a small few, it is clear that the future rests with developing the descendent of the R-L Monitor System. This system has always been known as P?S/8, a pun on the duality of a) The members of the PDP-8 programming world at large that met at Brooklyn Poly calling themselves the Poly Question Society [always abbreviated P?S if possible, or else PQS if in a limited character set situation], b) DEC's marketing penchant for forming product names suggestive of some ethereal system coupled with a suffix for the machine number.

Note: Certain members of the P?S submitted the R-L Monitor System to DECUS using the name MS/8 to further mock DEC marketing tactics. The R-L Monitor System can only run on a single TC01/TC08/TU55/TU56 DECTape drive unit [0]. If an image of this system is made available as a .tu56 SIMH standard image, it should run under the SIMH PDP-8 simulator.

Development of P?S/8 and some OS/8 issues.

P?S/8 is all new work based on the concepts started in the excellent base system that is the R-L Monitor System. The R-L Monitor System was created under incredible conditions, literally being hand-assembled code that was manually toggled into the PDP-8 using nothing other than the PDP-8 front panel and various sets of human fingers over a course of months.

Note: Charles Lasner, the principle author of P?S/8, this package and this document, personally repaired the computer subjected to this abuse. Literally, every front-panel slide switch was carefully replaced as they all had become quite unreliable; these switches are simply not designed for this class of rough service.

P?S/8 is still under development as of the writing of this document. As with the R-L Monitor before it, DEC managers were given two opportunities to review P?S/8 for the purpose of DEC taking it over for the "price" of hardware donation to Brooklyn Polytech for the benefit of students.

Appendix A [continued].

Note: Many large corporations routinely do this to lower their taxes, thus the actual cost to DEC conceivably could have been negligible or actually profitable. At the time of these "negotiations", members of the P?S still visited the Brooklyn Polytech facility unofficially, giving advice and guidance to yet another generation of programmers. Charles Lasner and others are considered the "second generation" of PDP-8 programmers who learned how to program the PDP-8 using the R-L monitor system instead of having to abuse the front panel. As such, they had an opportunity to critique the R-L Monitor System and then create improvements and new features over the course of many years of intermittent development.

P?S/8 was never written as a paid project; programmers have to seek employment by writing applications, not operating systems; by and large, clients were unaware of exactly how these projects got developed; impetus for some of the P?S/8 development stemmed from the need to accomplish work commitments to clients because there was no other way to accomplish what was required. For certain DEC employees, writing an operating system component or two was the entirety of their work. [At times, there are advantages to working in "the real world".]

Earlier releases of this package contained only P?S/8 Version 8P released circa 1976. More development work was performed on P?S/8 for as much as a decade past the release of this earlier work.

New to this package is the release of P?S/8 Version 8Z for TC01/TC08/TU55/TU56. This is essentially the system released first on 23-Feb-1988 with some binary patches applied to fix several minor known bugs. A change log file will be kept on P?S/8 development starting in 2015; it is anticipated a newer release [Version 9A] will be available after some work on the source files has been integrated into the general release.

At some point, work on the basic P?S/8 system will virtually cease, having taken it as far as it can go short of adding additional system programs [which will continue to be added].

The future of P?S/8 is the inclusion of an optional SHELL overlay which will be geared towards larger memory computers than the basic 4K required by the R-L Monitor System and P?S/8 as in the current implementation.

Note: P?S/8 requires only 4K; however, certain system programs require more memory to either improve performance, add additional capabilities or both, most notably P?S/8 PAL which can utilize as much as 20K of memory to support humongous assemblies with huge symbol tables. Few and far between one encounters PDP-8 projects that need that much memory to assemble the programs; the actual working binary programs could require up to 32K regardless of how many user symbols are in the source code.

Appendix A [continued].

Certain P?S/8 configurations require extended memory to operate due to the limitations of certain devices used to create the system resident kernel. Within the first 4K of memory, the requirements are met in all systems. However, certain system device handlers require an additional section of code resident in the highest available memory field [which must be in the range of field 1 through field 7]. However, assuming more than 8K is available, any user or system program is allowed to allocate 100% of the extended memory below the field of the handler extension without restrictions [unlike OS/8 which always reserves the last page of every field due to poor design].

The SHELL will be familiar to any seasoned DEC user; the user interface goals are very much like DEC systems created after OS/8 where the notion of CCL is not considered an option [that might have to be disabled].

Note: OS/8 is slavishly designed to match the capabilities of TOPS-10 from the days before CCL was created. CCL was added to TOPS-10 and then was sloppily added to OS/8 in a way that theoretically seems appropriate. Problems stem from the high-overhead process of software faking out other software which is profoundly inefficient and unnecessary in an alternate [better thought out] design. It is rather curious that the use of CCL [not necessarily named CCL, sometimes known as DCL] was universally encouraged within DEC with the lone exception of OS/8: Unless the system device was powerful enough, it was recommended CCL be disabled due to the serious overhead when used on common slower devices such as DECTape, a configuration that P?S/8 thrives on due to superior design.

The P?S/8 SHELL will allow CCL-like commands to be entered directly without the need for a clumsy intermediate command-decode mode. The goal will be performance first and slavish compatibility second, although the command structure will be quite familiar to anyone with a background in any DEC system based on either CCL or DCL.

Note: For backward-compatibility, the decode function will be used in the very few programs [such as PIP] where it makes sense to do so. However, the function may be so specifically designed, each program requiring a command mode may implement its own variant; it is likely few programs will completely agree upon specific requirements. However, the SHELL implementation will include support for various forms of commands not necessarily designed to be compatible with the SHELL file system conventions. This will allow "foreign" file systems to be supported, along with other variations known to be required by certain programs such as passing option strings not syntactically processed by the command decoder.

Appendix A [continued].

In some cases, OS/8 has some awkward usage of the command-decode function which was not designed to provide what is required for certain specific programs; bad kludges only allow barely adequate functionality because the command-decode function has arbitrary rules regarding command formation contrary to the needs of certain programs calling it. The alternative would be to "reinvent the wheel" which would point out the futility of the design. A heavy-handed approach was used to achieve a program that barely worked with unnecessary restrictions and high overhead.

Note: In the P?S/8 SHELL environment, this sort of inappropriate mechanism will be unnecessary as commands can be processed in various ways to suit the needs of a variety of utilities; much of the basic P?S/8 command processor already functions in this manner; the SHELL will implement a superset of what the basic system already supports; for commands where OS/8 struggles, P?S/8 already has several analogous commands implemented with little to no overhead.

The following is an example of awkward OS/8 command implementation:

The RX02 double-density diskette controller requires a rewrite of the data sectors to change the diskette functionality from single to double density or vice versa. This is sometimes erroneously referred to as a *format* operation. In actuality, true formatting is impossible on all DEC RX01/02/03 family drives due to hardware limitations.

Note: The third-party DSD-210 drive is RX01-compatible and can also format using an operating mode not used on the DEC RX01 accessed with special software. The superior quality of the diskette drives used in the DSD-210 allow for direct formatting; DEC used drives with excessive jitter that can work with formatted media, but it was deemed unacceptable to allow the drive to format magnetically blank media.

The OS/8 implementation of a program to accomplish the density-change function on the RX02 uses the standard command decoder merely to select the drive unit for the operation. Due to the overhead associated with the command decoder, the system must support the relevant non-system handlers merely to allow the command decoder the ability to parse the command line to determine the user's intention. The non-system handler plays no actual role in the program; however the only way for the command to work requires all the overhead associated with device and file open operations; the only information actually needed is the logical RX02 drive and the new density [single or double].

The CCL form of this command adds additional overhead to this already inefficient program that could have been implemented without the inflexible standard command decoder. Otherwise valid commands will fail merely because the current system doesn't support the virtually superfluous non-system device handler.

Appendix A [continued].

Note: OS/8 considers each logical unit of the same hardware device as a different software device. Unless the system configuration supports both the RXA0: and RXA1: non-system handlers, commands might fail despite the fact the handler itself is never called. However, all of the overhead associated with fetching the handler is required to parse the command.

Since the P?S/8 SHELL overlay is all new work, features equivalent to the above can be implemented with far less overhead.

Internally, the P?S/8 SHELL overlay will provide an environment analogous to what OS/8 provides to a system program. No attempt at absolute compatibility is to be implied here; analogous methods will be used to avoid limitations of OS/8 locked into the original design. However, adapting OS/8 system programs to the SHELL environment will be relatively straight-forward; it is expected that each and every useful OS/8 program will be ported to the P?S/8 SHELL environment eventually.

Because the SHELL design is open-ended and will implement much of the familiar OS/8 functionality [with the known quirks and limitations simply not present], the SHELL environment will be a complete logical replacement for OS/8 when completed; there will simply be no reason to run OS/8 as it will be inferior in most aspects and at best will be a poor second-rate system by comparison. An important consideration is to utilize memory better than OS/8. As such, P?S/8 should be superior to any imaginable OS/8 configuration. And yet, it will be an option which can be disabled yielding the present definition of the basic P?S/8 system.

The P?S/8 SHELL may or may not appear in P?S/8 Version 9A; the primary effort is to clean up the code to implement many small changes that are prerequisite to adding the shell design. In the interest of having a new cleaner implementation of the basic system, Version 9A may be the last P?S/8 release without the SHELL overlay; this will be decided some time in the future; as required, documentation of this package will be updated accordingly.

End of Appendix A.

[end-of-file]